PHENIKAA UNIVERSITY

PHENIKAA SCHOOL OF COMPUTING

COURSE: SOFTWARE ARCHITECTURE

HOTEL MANAGEMENT SYSTEM

INSTRUCTOR: Vũ Quang Dũng

GROUP 7: Lê Mạnh Đức (23010208 – K17-KTPM)

CREDIT CLASS: CSE703110-1-2-25(N01)

Ha Noi, March 6, 2026

Contents

1. Executive Summary

This project implements a Hotel Booking Management System that supports the end-to-end workflow of a real hotel: customers can browse rooms, create bookings, make payments, manage their wallet, and view booking history; receptionists can check guests in and out and manage service requests; administrators can manage rooms, services, staff, coupons, and reports. The system is architected using a layered architecture on top of Flask, with clear separation between controllers (HTTP layer), services (business logic), repositories (data access), and models (domain entities).

The backend lives under SRC/Arch and exposes REST-style JSON APIs (e.g. /api/bookings, /api/rooms, /api/auth/login, /api/wallet) through controller classes such as booking_controller.py, room_controller.py, user_controller.py, payment_controller.py, coupon_controller.py, checkin_controller.py, staff_controller.py, report_controller.py, and wallet_controller.py. These controllers rely on service classes (e.g. booking_service.py, room_service.py, payment_service.py, coupon_service.py) that encapsulate business logic and call repository classes (e.g. booking_repository.py, room_repository.py, coupon_repository.py) to access a MySQL database.

The frontend is implemented in SRC/UI as a set of HTML/CSS/JS pages for different roles: public pages for browsing rooms, authentication pages for login and registration, user pages for dashboard, my bookings, payment, and wallet, and admin pages for room and staff management and reports. The final outcome is a working multi-role hotel management application where the architecture supports modifiability, maintainability, and basic security and scalability for small-to-medium usage.

2. Project Requirements & Goals

2.1 Core Functional Requirements

The primary entities that interact with the Hotel Management System are:
1. User
2. Administrator
3. Receptionist

These requirements define the specific behaviors and functions the system must provide.

a) User Functional Requirements

ID	Requirement Name	Description	Priority

FR-01	Room Search & Booking	The User must be able to browse/search rooms by type and status, and complete a reservation online. Availability by date range is validated during booking creation (check-in/check-out overlap prevention).	High
FR-02	User Authentication	The User must be able to register and log in securely.	High
FR-03	Booking Management	The User must be able to view and cancel their reservations. The system must store a full booking history, while operational booking updates (e.g., status adjustments) are handled by receptionist/administrator roles.	High
FR-04	Online Payment	The User must be able to make secure online payments for reservations and additional services (spa, dining).	High
FR-05	Service Requests	The User must be able to request additional services during their stay.	Medium
FR-06	Digital Wallet	The User must be able to view their wallet balance, top up the wallet with a specified amount, and pay for bookings and services using the internal wallet balance.	High
FR-07	Coupon Application	The User must be able to apply valid promotional coupon codes during the booking or payment process to receive a discount (percentage or fixed amount). The system must validate coupon code, active status, and validity period.	Medium
FR-08	Contact with receptionist/admin	The User must be able to contact the receptionist or administrator (e.g., via the Contact/Support pages) to request services or ask for more information about rooms.	Medium

b) Receptionist Functional Requirements

ID	**Requirement Name**	**Description**	**Priority**

FR-09	Check-in/Check-out Processing	The Receptionist must be able to process guest check-in and check-out, including verifying bookings.	High
FR-10	Walk-in Booking	The Receptionist must be able to confirm bookings for guests arriving directly at the counter.	High
FR-11	Room Assignment	The Receptionist must be able to assign rooms to bookings based on room status and guest preferences.	High
FR-12	Transaction & Activity History	The Receptionist must be able to view historical records of bookings, payments, service requests, and system activities.	High
FR-13	Service Request Management	The Receptionist must be able to view the list of all service requests (e.g. room service, spa, dining) and update the status of each request (e.g. pending, in progress, completed).	High
FR-14	Refund on Cancellation	When a User cancels a booking that was paid (in whole or in part) using the digital wallet, the system must automatically refund the corresponding amount to the User's wallet and update the payment status (e.g. to "refunded"). Refund must only apply to eligible bookings (e.g. not yet checked in).	High
FR-15	Contact with guest	The Receptionist must be able to reply to guest questions and messages through the contact/support channels.	High

c) Hotel Manager Functional Requirements (Administrator)

ID	**Requirement Name**	**Description**	**Priority**
FR-16	Room Inventory Management	The Administrator must be able to perform full CRUD operations on the room catalog (add new rooms, update details, price) and manage room status (e.g., Under Maintenance).	High

FR-17	Staff Management	The Administrator must be able to create and manage staff accounts, including assigning roles (e.g., Receptionist, Housekeeper).	High
FR-18	Users Management	The Administrator must be able to see and manage users accounts.	High
FR-19	Service Management	The Administrator must be able to update the catalog of hotel services (Spa, Dining menu)	Medium
FR-20	Analytics & Reporting	The Administrator must have access to detailed reports on occupancy rates, revenue, and service performance.	Medium

2.2 Key Quality Attributes (Architectural Goals – Non-functional Requirements)

ID	Attribute	Description
NFR-01	Modifiability	The system is organized into layers (controllers, services, repositories, models) so that new features such as new payment methods, extended reports, or additional service types can be added with minimal impact on existing logic.
NFR-02	Maintainability	Code is grouped by responsibility (e.g. SRC/Arch/controllers, SRC/Arch/services, SRC/Arch/repository, SRC/Arch/models, SRC/Arch/views, SRC/Arch/middleware) providing clear entry points for debugging and enhancements.
NFR-03	Reliability & Consistency	Booking and payment logic (in booking_service.py, booking_repository.py, payment_service.py, and related repositories) must avoid overbooking, maintain correct booking status transitions, and ensure payment records match wallet balances.
NFR-04	Security	Authentication and role-based authorization are enforced via middleware (require_auth, require_admin, require_receptionist_or_admin, require_role) to secure sensitive endpoints such as booking, payment, wallet, and admin operations.

NFR-05	Usability	The UI under SRC/UI is designed to be clear and role-oriented: customers see booking and payment pages; staff see check-in/check-out and service request screens; admins see management and reporting screens.
NFR-06	Scalability	Although the current deployment is monolithic (single Flask app and a MySQL database), the RESTful API design and separation of services/repositories allows future scaling by splitting out services or deploying behind a load balancer.
NFR-07	Performance	The architecture utilizes direct function calls between layers (Controller → Service → Repository) eliminating internal network hops. Database queries are optimized with indexing.

2.3 Propose model of ASR

ASR ID	Quality Attribute	Statement	Impact
ASR-1	Prevent Overbooking (NFR – Reliability & Consistency)	The system must ensure that a specific room cannot be booked by multiple users for overlapping	This requirement directly shapes the design of booking_service.py and booking_repository.py. For example, booking_service.py uses room_service.py which checks the database for existing bookings with

		date ranges.	overlapping check-in/check-out dates and excludes such rooms from the available list. The SQL query in booking_repository.py filters out rooms whose bookings overlap in time and are not cancelled or checked_out, preventing double-booking at the data-access level.
ASR-2	Secure Access to Financial Operations (NFR – Security & Integrity)	All wallet and payment-related operations must be performed only by authenticated users, and only on their own accounts or authorized resources..	This requirement leads to the use of authentication middleware and role-based decorators around endpoints in app.py such as /api/payments, /api/wallet, /api/wallet/topup, and admin-only routes. payment_service.py and wallet_service.py must always validate the current user's identity and ensure that wallet updates and payment records are tied to the correct user_id and booking_id, preventing unauthorized manipulation of balances or payments.
ASR-3	Layered Modifiability (NFR – Modifiability)	New business rules (e.g., coupon rules, booking constraints, new payment methods) must be implemented primarily in the Service layer without changing the REST endpoints and without	This requirement is implemented by the layered structure in SRC/Arch: controllers parse HTTP input and delegate to services; services encapsulate business logic; repositories encapsulate SQL. This reduces change impact and keeps changes localized.

		spreading SQL into controllers.	
ASR-4	Maintainable Code Organization & Clear Entry Points (NFR – Maintainability)	The codebase must be organized by responsibility so developers can quickly locate where to add validations, debug failures, or extend features.	This requirement is satisfied by the directory structure and separation of concerns: SRC/Arch/controllers (HTTP), SRC/Arch/services (business rules), SRC/Arch/repository (SQL), SRC/Arch/models (entities), SRC/Arch/views (JSON formatting), SRC/Arch/middleware (RBAC), SRC/Arch/utils (utilities). For example, auth issues are investigated in middleware/auth.py; booking logic in services/booking_service.py; SQL overlap logic in repository/booking_repository.py.
ASR-5	Consistent Error Responses Across APIs (NFR – Reliability & Maintainability)	All endpoints must return consistent JSON error payloads and appropriate HTTP status codes so the frontend can handle failures uniformly.	This requirement is reflected by controller + view patterns: controllers catch ValueError and return view.error_response(message, status) with 400/401/403/404, while unexpected errors map to 500. The UI-side API wrapper (SRC/UI/js/api.js) expects JSON errors and handles 401 by clearing currentUser. This reduces frontend complexity and improves reliability.
ASR-6	Query Performance for Search and Availability (NFR – Performance)	Room search and date-range availability checks must remain responsive under	This requirement is supported by database indexes defined in Design/database_schema.sql, such as indexes on rooms(room_type, status) and bookings(check_in_date, check_out_date), plus

		typical loads by relying on indexed queries for common filters (room_type, status, date range).	booking/payment indexes for lookups. The availability query in booking_repository.py/ BookingRepository.find_available_rooms_by_date_range(...) is designed to leverage these indexes to filter rooms and exclude overlapping bookings efficiently.
ASR-7	Stateless API for Horizontal Scaling (NFR – Scalability)	The backend APIs should remain largely stateless so the monolith can be replicated behind a load balancer in the future (without server-side session affinity requirements).	This requirement is supported by the REST-style API and header-based identity propagation (X-User-Id) combined with database persistence. Since user context is derived from each request header via middleware, the Flask process can be scaled out by running multiple instances as long as they share the same MySQL database and upload storage strategy.

Architectural tactics:

ASR-1:

- Enforce room availability checks at the service layer (BookingService and RoomService) before creating or updating a booking.
- Use a single SQL overlap query in booking_repository.py/BookingRepository.find_available_rooms_by_date_range(...) to centralize consistency checks and execute them inside a database transaction.

ASR-2:

- Apply authentication and role-based authorization middleware (require_auth, require_admin, require_receptionist_or_admin) on all financial endpoints before business logic is executed.
- Validate user identity and resource ownership inside PaymentService and WalletService by checking user_id against booking_id and wallet records before updating balances or payment status.

ASR-3:

- Enforce a strict layering rule (Controller → Service → Repository → Database) where controllers are thin HTTP adapters and all business rules live in services.
- Isolate SQL access in repository classes so that new or changed rules are implemented by extending services without modifying controllers or duplicating SQL.

ASR-4:

- Organize files by technical responsibility (controllers, services, repositories, models, views, middleware, utils) rather than by pages, so each concern has a clear "home".
- Use consistent naming and domain slices (e.g., booking_controller.py, booking_service.py, booking_repository.py, booking.py, booking_view.py) to make navigation and change-impact analysis straightforward.

ASR-5:

- Centralize error formatting in view classes (e.g., BookingView.error_response) so controllers always return structured JSON with HTTP status codes.
- Use a convention where services raise ValueError for business rule violations and controllers catch them to map to 4xx responses, while unexpected exceptions propagate to generic 500 handlers.

ASR-6:

- Create database indexes on frequently filtered columns (e.g., rooms.room_type, rooms.status, bookings.check_in_date, bookings.check_out_date, and foreign keys used by booking and payment queries).
- Use a dedicated availability query in BookingRepository.find_available_rooms_by_date_range(...) that performs filtering in the database instead of loading all rooms into memory.

ASR-7:

- Keep the Flask API stateless by deriving user context from each request's X-User-Id header and persisting all long-lived state in MySQL and uploads storage rather than in server-side sessions.
- Design REST endpoints so that multiple Flask processes behind a load balancer can handle requests interchangeably as long as they share the same database and uploads directory.